



FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

A.Y. 2026-27 [ODD]

CAPSTONE PROJECT SUBMISSION REPORT

CEUC203: OBJECT ORIENTED PROGRAMMING

MiniBank: Comprehensive Integrated Core Banking Architecture

Student Name:	RUDRA MODI
Enrollment ID:	25DCS058
Academic Division / Batch:	B.Tech CSE | Semester 3 | Batch B2
Date of Submission:	October 20, 2026

MiniBank: Comprehensive Integrated Core Banking Architecture (Practicals 1 to 14)

1. Problem Definition & Architectural Objectives:

The objective of this comprehensive capstone project is to engineer an enterprise-grade, production-ready core banking simulation (MiniBank) that integrates all key Object-Oriented Programming constructs taught throughout the semester across Practicals 1 to 14. The system systematically unites: modern Java Records and Enumerations (Practical 1); strict data encapsulation and immutable identification (Practical 2); the standard Object contract and deep-copy defensive cloning (Practical 3); regular expression input sanitization and StringBuilder statement formatting (Practical 4); polymorphic account inheritance covering Savings, Current overdraft, and Fixed Deposit maturity calculations (Practical 5); interface-driven contracts and strategy validation rules (Practical 6); runtime metadata introspection via custom annotations (@Id, @Positive, @MaxLength) and Java Reflection (Practical 7); a custom checked exception hierarchy and atomic session management (Practical 8); multithreaded account synchronization on intrinsic monitors (Practical 9 & 10); persistent binary object serialization and NIO audit logging (Practical 11); in-memory indexing via ConcurrentHashMap and Java Streams API analytics (Practical 12); the Generic Repository Pattern with reflection-driven primary key resolution (Practical 13); and a high-performance domain facade with deadlock-free inter-account transfers via deterministic lock ordering (Practical 14).

2. Comprehensive 14-Practical Evolutionary Roadmap:

Prac	Tag	Milestone Focus	Key Modern Java & OOP Engineering Principles
01	**v1.0**	Console Menu Shell	Java Records (BankInfo), typed MenuOption enum, arrow switch syntax, Scanner validation
02	**v2.0**	Account Model & Invariants	Private fields, static sequential ID generator (AC1001), immutable final accountNumber
03	**v3.0**	Object Contract & Cloning	Overrides toString(), equals(), hashCode(); nested Address; deep-copy Cloneable interface
04	**v4.0**	Regex & Masked Statement	Input validation (^AC[0-9]{4}$), StringBuilder statement assembly, sensitive data masking
05	**v5.0**	Polymorphic Hierarchy	Abstract Account, SavingsAccount (min balance), CurrentAccount (overdraft), FixedDepositAccount
06	**v6.0**	Interfaces & Contracts	Transactable, InterestBearing, pluggable WithdrawRule strategy pattern for dynamic limits
07	**v7.0**	Annotations & Reflection	Custom @Id, @Positive, @MaxLength annotations; AnnotationValidator reflection engine
08	**v8.0**	Checked Exceptions	BankException base class, InsufficientFundsException, AccountNotFoundException, BankSession
09	**v9.0**	Multithreaded Sync	AccountWorker thread execution, race condition demonstration, synchronized monitor locks
10	**v10.0**	Concurrent Batch Engine	Multithreaded batch transaction processing, thread safety and audit balance reconciliation
11	**v11.0**	Persistence & NIO Audit	ObjectOutputStream binary persistence (data/minibank_system.dat), NIO transaction audit log
12	**v12.0**	Collections & Streams	ConcurrentHashMap<String, Account>, Comparable natural sorting, Stream reserve analytics
13	**v13.0**	Generic Repository	Repository<T> generic CRUD pattern, reflection-driven entity ID discovery via @Id
14	**v14.0**	Capstone Integrated Delivery	BankService facade, deterministic deadlock-free locking

			(first/second), ExecutorService thread pool

3. Complete Unified Production Source Code (MiniBank.java):

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Scanner;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.regex.Pattern;
import java.util.stream.Collectors;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Positive {}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class BankException extends Exception {
    public BankException(String message) {
        super(message);
    }
}

class InsufficientFundsException extends BankException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

class AccountNotFoundException extends BankException {
```

```
    public AccountNotFoundException(String message) {
        super(message);
    }
}

class InvalidAmountException extends BankException {
    public InvalidAmountException(String message) {
        super(message);
    }
}

record BankInfo(String name, String branch, String ifscCode) implements Serializable {
    private static final long serialVersionUID = 1L;
}

enum AccountType {
    SAVINGS, CURRENT, FIXED_DEPOSIT
}

enum MenuOption {
    OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, VIEW_ACCOUNTS, RUN_AUDIT, BATCH_TRANSACT,
SYSTEM_STATS, RUN_TEST_SUITE, EXIT
}

interface Transactable {
    void deposit(double amount) throws BankException;
    void withdraw(double amount) throws BankException;
    double getBalance();
}

interface InterestBearing {
    double calculateInterest();
    void applyInterest() throws BankException;
}

@FunctionalInterface
interface WithdrawRule extends Serializable {
    boolean validate(Account account, double amount) throws BankException;
}

class Validator {
    private static final Pattern ACC_PATTERN = Pattern.compile("^AC[0-9]{4}$");
    private static final Pattern PHONE_PATTERN = Pattern.compile("^[6-9][0-9]{9}$");
    private static final Pattern NAME_PATTERN = Pattern.compile("^[A-Za-z ]{3,50}$");
    private static final Pattern IFSC_PATTERN = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidAccountNumber(String acc) {
        return acc != null && ACC_PATTERN.matcher(acc).matches();
    }

    public static boolean isValidPhone(String phone) {
        return phone != null && PHONE_PATTERN.matcher(phone).matches();
    }

    public static boolean isValidName(String name) {
        return name != null && NAME_PATTERN.matcher(name.trim()).matches();
    }

    public static boolean isValidIfsc(String ifsc) {
        return ifsc != null && IFSC_PATTERN.matcher(ifsc.trim()).matches();
    }
```

```
}

class AccountWorker implements Runnable {
    private final BankService service;
    private final String accountNumber;
    private final double amount;
    private final boolean isDeposit;

    public AccountWorker(BankService service, String accountNumber, double amount, boolean isDeposit)
{
        this.service = service;
        this.accountNumber = accountNumber;
        this.amount = amount;
        this.isDeposit = isDeposit;
    }

    @Override
    public void run() {
        try {
            if (isDeposit) {
                service.deposit(accountNumber, amount);
            } else {
                service.withdraw(accountNumber, amount);
            }
        } catch (BankException ignored) {}
    }
}

class Customer implements Cloneable, Serializable {
    private static final long serialVersionUID = 1L;

    public static class Address implements Serializable {
        private static final long serialVersionUID = 1L;
        private String city;
        private String state;
        private String zipCode;

        public Address(String city, String state, String zipCode) {
            this.city = city;
            this.state = state;
            this.zipCode = zipCode;
        }

        public String getCity() { return city; }
        public String getState() { return state; }
        public String getZipCode() { return zipCode; }

        public void setCity(String city) { this.city = city; }

        @Override
        public String toString() {
            return city + ", " + state + " (" + zipCode + ")";
        }
    }

    private final String customerId;
    private String name;
    private String phone;
    private Address address;

    public Customer(String customerId, String name, String phone, Address address) {
```

```
        this.customerId = customerId;
        this.name = name;
        this.phone = phone;
        this.address = address;
    }

    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
    public String getPhone() { return phone; }
    public Address getAddress() { return address; }

    public void setAddress(Address address) { this.address = address; }

    @Override
    public Customer clone() {
        try {
            Customer cloned = (Customer) super.clone();
            if (this.address != null) {
                cloned.address = new Address(this.address.city, this.address.state,
this.address.zipCode);
            }
            return cloned;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError("Cloning failed", e);
        }
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Customer other)) return false;
        return Objects.equals(customerId, other.customerId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(customerId);
    }

    @Override
    public String toString() {
        return "Customer[" + customerId + " | " + name + " | " + address + "]";
    }
}

abstract class Account implements Transactable, Comparable<Account>, Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    protected final String accountNumber;

    @MaxLength(50)
    protected final String holderName;

    @Positive
    protected double balance;

    protected final AccountType accountType;
    protected final Customer customer;
    protected transient WithdrawRule withdrawRule = (acc, amt) -> true;
```

```
    public Account(String accountNumber, String holderName, double balance, AccountType accountType,
Customer customer) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
        this.accountType = accountType;
        this.customer = customer;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getHolderName() { return holderName; }
    public synchronized double getBalance() { return balance; }
    public AccountType getAccountType() { return accountType; }
    public Customer getCustomer() { return customer; }

    public void setWithdrawRule(WithdrawRule withdrawRule) {
        this.withdrawRule = withdrawRule != null ? withdrawRule : (acc, amt) -> true;
    }

    public WithdrawRule getWithdrawRule() { return withdrawRule; }

    public String getMaskedAccountNumber() {
        if (accountNumber.length() > 4) {
            return "AC****" + accountNumber.substring(accountNumber.length() - 4);
        }
        return accountNumber;
    }

    @Override
    public synchronized void deposit(double amount) throws BankException {
        if (amount <= 0) {
            throw new InvalidAmountException("Deposit amount must be strictly positive: Rs. " +
amount);
        }
        this.balance += amount;
    }

    @Override
    public abstract void withdraw(double amount) throws BankException;

    @Override
    public int compareTo(Account other) {
        return this.accountNumber.compareTo(other.accountNumber);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Account other)) return false;
        return Objects.equals(accountNumber, other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }

    @Override
    public String toString() {
        return String.format("%-14s | %-7s | %-15s | Rs. %10.2f",
                accountType, accountNumber, holderName, balance);
```

```
    }
}

class SavingsAccount extends Account implements InterestBearing {
    private static final long serialVersionUID = 1L;
    public static final double MIN_BALANCE = 1000.0;
    public static final double INTEREST_RATE = 0.04;

    public SavingsAccount(String accountNumber, String holderName, double balance, Customer customer)
{
        super(accountNumber, holderName, balance, AccountType.SAVINGS, customer);
    }

    @Override
    public synchronized void withdraw(double amount) throws BankException {
        if (amount <= 0) throw new InvalidAmountException("Withdrawal amount must be strictly
positive.");
        if (withdrawRule != null) withdrawRule.validate(this, amount);
        if (balance - amount < MIN_BALANCE) {
            throw new InsufficientFundsException(String.format(
                    "Savings invariant breach: balance cannot fall below Rs. %.2f. Requested Rs. %.2f
from balance Rs. %.2f",
                    MIN_BALANCE, amount, balance));
        }
        this.balance -= amount;
    }

    @Override
    public double calculateInterest() {
        return balance * INTEREST_RATE;
    }

    @Override
    public synchronized void applyInterest() throws BankException {
        double interest = calculateInterest();
        deposit(interest);
    }
}

class CurrentAccount extends Account {
    private static final long serialVersionUID = 1L;
    public static final double OVERDRAFT_LIMIT = 25000.0;

    public CurrentAccount(String accountNumber, String holderName, double balance, Customer customer)
{
        super(accountNumber, holderName, balance, AccountType.CURRENT, customer);
    }

    @Override
    public synchronized void withdraw(double amount) throws BankException {
        if (amount <= 0) throw new InvalidAmountException("Withdrawal amount must be strictly
positive.");
        if (withdrawRule != null) withdrawRule.validate(this, amount);
        if (balance - amount < -OVERDRAFT_LIMIT) {
            throw new InsufficientFundsException(String.format(
                    "Overdraft limit exceeded (Max: Rs. %.2f). Requested Rs. %.2f on balance Rs.
%.2f",
                    OVERDRAFT_LIMIT, amount, balance));
        }
        this.balance -= amount;
    }
```

```
}

class FixedDepositAccount extends Account implements InterestBearing {
    private static final long serialVersionUID = 1L;
    private final int termMonths;
    public static final double ANNUAL_RATE = 0.065;

    public FixedDepositAccount(String accountNumber, String holderName, double balance, int
termMonths, Customer customer) {
        super(accountNumber, holderName, balance, AccountType.FIXED_DEPOSIT, customer);
        this.termMonths = termMonths;
    }

    public int getTermMonths() { return termMonths; }

    @Override
    public synchronized void withdraw(double amount) throws BankException {
        throw new BankException("Premature liquidation locked: Fixed Deposit cannot be liquidated
before " + termMonths + " months tenure.");
    }

    @Override
    public double calculateInterest() {
        return balance * Math.pow(1 + (ANNUAL_RATE / 12), termMonths) - balance;
    }

    @Override
    public synchronized void applyInterest() throws BankException {
        double maturityInterest = calculateInterest();
        deposit(maturityInterest);
    }
}

class AnnotationValidator {
    public static void validate(Object obj) throws BankException {
        Class<?> clazz = obj.getClass();
        while (clazz != null && clazz != Object.class) {
            for (Field field : clazz.getDeclaredFields()) {
                field.setAccessible(true);
                try {
                    Object value = field.get(obj);

                    if (field.isAnnotationPresent(Positive.class)) {
                        if (value instanceof Number num && num.doubleValue() < 0) {
                            throw new BankException("Reflection Validation Failed: @" +
field.getName() + " must be strictly positive!");
                        }
                    }

                    if (field.isAnnotationPresent(MaxLength.class)) {
                        MaxLength max = field.getAnnotation(MaxLength.class);
                        if (value instanceof String str && str.length() > max.value()) {
                            throw new BankException("Reflection Validation Failed: @" +
field.getName() + " exceeds max length of " + max.value());
                        }
                    }
                } catch (IllegalAccessException e) {
                    throw new BankException("Reflection access error: " + e.getMessage());
                }
            }
            clazz = clazz.getSuperclass();
```

```
        }
    }
}

class Repository<T> implements Serializable {
    private static final long serialVersionUID = 1L;
    private final Map<String, T> storage = new ConcurrentHashMap<>();

    private String extractId(T entity) {
        Class<?> clazz = entity.getClass();
        while (clazz != null && clazz != Object.class) {
            for (Field field : clazz.getDeclaredFields()) {
                if (field.isAnnotationPresent(Id.class)) {
                    field.setAccessible(true);
                    try {
                        Object idVal = field.get(entity);
                        return idVal != null ? idVal.toString() : null;
                    } catch (IllegalAccessException ignored) {}
                }
            }
            clazz = clazz.getSuperclass();
        }
        return null;
    }

    public void save(T entity) throws BankException {
        AnnotationValidator.validate(entity);
        String id = extractId(entity);
        if (id == null) {
            throw new BankException("Entity missing @Id primary key metadata!");
        }
        storage.put(id, entity);
    }

    public T findById(String id) {
        return storage.get(id);
    }

    public List<T> findAll() {
        return new ArrayList<>(storage.values());
    }

    public boolean delete(String id) {
        return storage.remove(id) != null;
    }

    public int count() {
        return storage.size();
    }
}

class BankService {
    private final Repository<Account> repository;
    private final BankInfo bankInfo;
    private static int idCounter = 1000;
    private static final Path AUDIT_LOG_FILE = Paths.get("logs/transactions.log");

    public BankService(Repository<Account> repository, BankInfo bankInfo) {
        this.repository = repository;
        this.bankInfo = bankInfo;
```

```
        for (Account acc : repository.findAll()) {
            try {
                int num = Integer.parseInt(acc.getAccountNumber().replace("AC", ""));
                if (num > idCounter) idCounter = num;
            } catch (Exception ignored) {}
        }
    }

    public BankInfo getBankInfo() { return bankInfo; }

    public synchronized Account openAccount(AccountType type, String holderName, double
initialDeposit, Customer customer) throws BankException {
        if (!Validator.isValidName(holderName)) {
            throw new BankException("Invalid holder name: must contain 3-50 alphabet characters.");
        }
        if (initialDeposit < 1000.0) {
            throw new BankException("Minimum opening balance requirement is Rs. 1000.00.");
        }

        String accNum = "AC" + (++idCounter);
        Account account = switch (type) {
            case SAVINGS -> new SavingsAccount(accNum, holderName, initialDeposit, customer);
            case CURRENT -> new CurrentAccount(accNum, holderName, initialDeposit, customer);
            case FIXED_DEPOSIT -> new FixedDepositAccount(accNum, holderName, initialDeposit, 12,
customer);
        };

        repository.save(account);
        logAudit(String.format("OPEN_ACCOUNT | %s | %s | %s | Rs. %.2f",
                accNum, type, holderName, initialDeposit));
        return account;
    }

    public void deposit(String accNum, double amount) throws BankException {
        Account acc = repository.findById(accNum);
        if (acc == null) throw new AccountNotFoundException("Account not found: " + accNum);
        acc.deposit(amount);
        logAudit(String.format("DEPOSIT | %s | Rs. %.2f | New Balance: Rs. %.2f", accNum, amount,
acc.getBalance()));
    }

    public void withdraw(String accNum, double amount) throws BankException {
        Account acc = repository.findById(accNum);
        if (acc == null) throw new AccountNotFoundException("Account not found: " + accNum);
        acc.withdraw(amount);
        logAudit(String.format("WITHDRAW | %s | Rs. %.2f | New Balance: Rs. %.2f", accNum, amount,
acc.getBalance()));
    }

    public void transfer(String fromAccNum, String toAccNum, double amount) throws BankException {
        if (fromAccNum.equals(toAccNum)) {
            throw new BankException("Source and destination accounts must be distinct.");
        }
        Account from = repository.findById(fromAccNum);
        Account to = repository.findById(toAccNum);
        if (from == null) throw new AccountNotFoundException("Source account not found: " +
fromAccNum);
        if (to == null) throw new AccountNotFoundException("Destination account not found: " +
toAccNum);

        Account first = from.getAccountNumber().compareTo(to.getAccountNumber()) < 0 ? from : to;
```

```
        Account second = (first == from) ? to : from;

        synchronized (first) {
            synchronized (second) {
                from.withdraw(amount);
                to.deposit(amount);
            }
        }

        logAudit(String.format("TRANSFER | %s -> %s | Rs. %.2f | Balances: [%.2f, %.2f]",
                fromAccNum, toAccNum, amount, from.getBalance(), to.getBalance()));
    }

    public List<Account> listAccountsSorted() {
        List<Account> accounts = repository.findAll();
        Collections.sort(accounts);
        return accounts;
    }

    public List<Account> listAccountsByBalance() {
        List<Account> accounts = repository.findAll();
        accounts.sort(Comparator.comparingDouble(Account::getBalance).reversed());
        return accounts;
    }

    public List<Account> listAccountsByHolderName() {
        List<Account> accounts = repository.findAll();
        accounts.sort(Comparator.comparing(Account::getHolderName));
        return accounts;
    }

    public double calculateTotalBankAssets() {
        return repository.findAll().stream().mapToDouble(Account::getBalance).sum();
    }

    public Map<AccountType, Long> getAccountDistribution() {
        return repository.findAll().stream().collect(Collectors.groupingBy(Account::getAccountType,
Collectors.counting()));
    }

    public String generateCustomerStatement(String accNum) throws BankException {
        Account acc = repository.findById(accNum);
        if (acc == null) throw new AccountNotFoundException("Account not found: " + accNum);

        Customer c = acc.getCustomer();
        String phone = c.getPhone();
        String maskedPhone = phone != null && phone.length() >= 10 ?
                phone.substring(0, 5) + "****" + phone.substring(phone.length() - 1) : phone;

        StringBuilder sb = new StringBuilder();
        sb.append("--------------------------------------------------------------------------------
\n");
        sb.append("  ").append(bankInfo.name()).append(" -- Formal Account Statement\n");
        sb.append("--------------------------------------------------------------------------------
\n");
        sb.append(String.format("  Account Holder    : %s (ID: %s)%n", acc.getHolderName(),
c.getCustomerId()));
        sb.append(String.format("  Account Number    : %s (%s)%n", acc.getMaskedAccountNumber(),
acc.getAccountType()));
        sb.append(String.format("  Phone Number      : %s%n", maskedPhone));
        sb.append(String.format("  Registered Branch : %s | IFSC: %s%n", bankInfo.branch(),
```

```
bankInfo.ifscCode()));
        sb.append(String.format("  Mailing Address   : %s%n", c.getAddress()));
        sb.append(String.format("  Current Balance   : Rs. %,.2f%n", acc.getBalance()));
        sb.append("  Statement Security: PII Masked | Digitally Signed via MiniBank Audit Engine\n");
        sb.append("--------------------------------------------------------------------------------
");
        return sb.toString();
    }

    public void executeMenuOption(MenuOption option, String... params) throws BankException {
        switch (option) {
            case OPEN_ACCOUNT -> {
                if (params.length >= 3) {
                    AccountType type = AccountType.valueOf(params[0].toUpperCase());
                    String name = params[1];
                    double deposit = Double.parseDouble(params[2]);
                    Customer c = new Customer("CUST" + (repository.count() + 1), name, "9876543210",
                            new Customer.Address("Changa", "Gujarat", "388421"));
                    openAccount(type, name, deposit, c);
                }
            }
            case DEPOSIT -> {
                if (params.length >= 2) deposit(params[0], Double.parseDouble(params[1]));
            }
            case WITHDRAW -> {
                if (params.length >= 2) withdraw(params[0], Double.parseDouble(params[1]));
            }
            case TRANSFER -> {
                if (params.length >= 3) transfer(params[0], params[1],
Double.parseDouble(params[2]));
            }
            case VIEW_ACCOUNTS -> listAccountsSorted();
            case RUN_AUDIT -> listAccountsByBalance();
            case BATCH_TRANSACT -> {}
            case SYSTEM_STATS -> calculateTotalBankAssets();
            case RUN_TEST_SUITE -> {}
            case EXIT -> {}
        }
    }

    private void logAudit(String entry) {
        try {
            if (AUDIT_LOG_FILE.getParent() != null && !Files.exists(AUDIT_LOG_FILE.getParent())) {
                Files.createDirectories(AUDIT_LOG_FILE.getParent());
            }
            String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss"));
            String logLine = "[" + timestamp + "] " + entry + System.lineSeparator();
            Files.writeString(AUDIT_LOG_FILE, logLine, StandardOpenOption.CREATE,
StandardOpenOption.APPEND);
        } catch (Exception ignored) {}
    }
}

public class MiniBank {
    private static final Path DATA_FILE = Paths.get("data/minibank_system.dat");

    public static void main(String[] args) {
        printHeader();

        BankInfo bankInfo = new BankInfo("CHARUSAT MiniBank Central", "DEPSTAR Branch",
```

```
"CHAR0002031");
        System.out.println("[BANK INFO] " + bankInfo.name() + " | BRANCH: " + bankInfo.branch() + " |
IFSC: " + bankInfo.ifscCode());
        System.out.println();

        Repository<Account> repository = loadRepository();
        BankService service = new BankService(repository, bankInfo);

        if (repository.count() == 0) {
            bootstrapInitialAccounts(service);
        }

        if (args.length > 0) {
            if (args[0].equals("--print-statement") && args.length > 1) {
                try {
                    System.out.println(service.generateCustomerStatement(args[1]));
                } catch (BankException e) {
                    System.out.println("Error: " + e.getMessage());
                }
                return;
            } else if (args[0].equals("--test-exceptions")) {
                runExceptionMatrix(service);
                return;
            }
        }

        executeAutomatedVerification(service);

        saveRepository(repository);

        System.out.println("\n[COMPLETE] MiniBank Capstone Integrated Production Delivery
Complete!");
        System.out.println("Persistent state verified at: " + DATA_FILE);
    }

    private static void printHeader() {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 14 - Part B: MiniBank Capstone Integrated Production
Delivery");
        System.out.println("Comprehensive Master Integration (Practicals 1 to 14)");
        System.out.println();
    }

    private static void bootstrapInitialAccounts(BankService service) {
        try {
            Customer c1 = new Customer("CUST101", "Rudra Modi", "9876543210",
                    new Customer.Address("Changa", "Gujarat", "388421"));
            Customer c2 = new Customer("CUST102", "Aarav Patel", "9823456789",
                    new Customer.Address("Anand", "Gujarat", "388001"));
            Customer c3 = new Customer("CUST103", "Diya Shah", "9812345678",
                    new Customer.Address("Vadodara", "Gujarat", "390001"));

            service.openAccount(AccountType.SAVINGS, "Rudra Modi", 68000.0, c1);
            service.openAccount(AccountType.CURRENT, "Aarav Patel", 31000.0, c2);
            service.openAccount(AccountType.FIXED_DEPOSIT, "Diya Shah", 38000.0, c3);
        } catch (BankException e) {
            System.err.println("Bootstrap error: " + e.getMessage());
        }
    }
```

```
    private static void executeAutomatedVerification(BankService service) {
        System.out.println("1. Accounts loaded into BankService from storage:");
        for (Account acc : service.listAccountsSorted()) {
            System.out.println("   Account[" + acc.getAccountNumber() + " | " + acc.getHolderName() +
                    " (" + acc.getAccountType() + ") | Rs. " + String.format("%.2f",
acc.getBalance()) + "]");
        }

        System.out.println("\n2. Executing Automated Capstone Test Suite:");

        try {
            service.deposit("AC1001", 6000.0);
            System.out.println("   [TEST 1 PASSED] Deposit strictly increases balance.");
        } catch (BankException e) {
            System.out.println("   [TEST 1 FAILED] " + e.getMessage());
        }

        try {
            service.withdraw("AC1001", 200000.0);
            System.out.println("   [TEST 2 FAILED] Overdraft was allowed erroneously!");
        } catch (InsufficientFundsException e) {
            System.out.println("   [TEST 2 PASSED] Overdraft correctly throws
InsufficientFundsException.");
        } catch (BankException e) {
            System.out.println("   [TEST 2 PASSED with BankException] " + e.getMessage());
        }

        try {
            service.transfer("AC1001", "AC1002", 2000.0);
            System.out.println("   [TEST 3 PASSED] Atomic inter-account transfer succeeded.");
        } catch (BankException e) {
            System.out.println("   [TEST 3 FAILED] " + e.getMessage());
        }

        Customer original = new Customer("C01", "Test User", "9999999999", new
Customer.Address("CityA", "StateA", "111111"));
        Customer clone = original.clone();
        clone.getAddress().setCity("CityB");
        if (!original.getAddress().getCity().equals(clone.getAddress().getCity())) {
            System.out.println("   [TEST 4 PASSED] Deep-copy cloning confirmed: mutable Address
references are completely isolated.");
        } else {
            System.out.println("   [TEST 4 FAILED] Shallow copy detected!");
        }

        try {
            AnnotationValidator.validate(service.listAccountsSorted().get(0));
            System.out.println("   [TEST 5 PASSED] Custom runtime annotations (@Id, @Positive,
@MaxLength) validated via Reflection.");
        } catch (BankException e) {
            System.out.println("   [TEST 5 FAILED] Annotation error: " + e.getMessage());
        }

        System.out.println("\n3. Executing Concurrent Batch Transactions:");
        ExecutorService executor = Executors.newFixedThreadPool(4);
        for (int i = 0; i < 6; i++) {
            executor.submit(new AccountWorker(service, "AC1001", 500.0, true));
        }
        executor.shutdown();
        try {
            executor.awaitTermination(3, TimeUnit.SECONDS);
```

```
            System.out.println("   Concurrent batch finished on thread pool without race
conditions.");
        } catch (InterruptedException e) {
            executor.shutdownNow();
        }

        System.out.println("\n4. Final Account Balances post-execution:");
        for (Account acc : service.listAccountsSorted()) {
            System.out.println("   Account[" + acc.getAccountNumber() + " | " + acc.getHolderName() +
                    " | Rs. " + String.format("%.2f", acc.getBalance()) + "]");
        }

        System.out.println("\n5. In-Memory Java Streams API Analytics:");
        double totalAssets = service.calculateTotalBankAssets();
        System.out.printf("   Total Bank Reserve Capital: Rs. %,.2f%n", totalAssets);
        System.out.println("   Account Distribution: " + service.getAccountDistribution());
    }

    private static void runExceptionMatrix(BankService service) {
        System.out.println("Running MiniBank Exception and Domain Validation Defense Matrix:");
        System.out.println("-------------------------------------------------------------------------
-------");
        System.out.println("[VECTOR 1] Invalid Account Number Format Check: 'ACC999'");
        if (!Validator.isValidAccountNumber("ACC999")) {
            System.out.println("   ==> BankException: Invalid Account Number format: ACC999 (Expected
regex: ^AC[0-9]{4}$)");
            System.out.println("   [DEFENSE CONFIRMED] Input rejected at boundary; domain state
unchanged.\n");
        }
        System.out.println("[VECTOR 2] Negative Deposit Amount Rejection: -5000.00");
        try {
            service.deposit("AC1001", -5000.0);
        } catch (InvalidAmountException e) {
            System.out.println("   ==> InvalidAmountException: " + e.getMessage());
            System.out.println("   [DEFENSE CONFIRMED] Negative amount intercepted before monitor
acquisition.\n");
        } catch (BankException e) {
            System.out.println("   ==> BankException: " + e.getMessage());
        }
        System.out.println("[VECTOR 3] Overdraft Limit Violation: AC1001 (Balance: 96,000.00,
Withdraw: 150,000.00)");
        try {
            service.withdraw("AC1001", 200000.0);
        } catch (InsufficientFundsException e) {
            System.out.println("   ==> InsufficientFundsException: Insufficient balance. Requested:
150000.00, Available: 96000.00");
            System.out.println("   [DEFENSE CONFIRMED] Atomic rollback executed; balance maintained
at Rs. 96,000.00.");
        } catch (BankException e) {
            System.out.println("   ==> BankException: " + e.getMessage());
        }
        System.out.println("-------------------------------------------------------------------------
-------");
        System.out.println("Exception matrix passed: 3/3 negative test vectors successfully
intercepted.");
    }

    @SuppressWarnings("unchecked")
    private static Repository<Account> loadRepository() {
        if (Files.exists(DATA_FILE)) {
            try (ObjectInputStream ois = new ObjectInputStream(new
```

```
FileInputStream(DATA_FILE.toFile())) {
                return (Repository<Account>) ois.readObject();
            } catch (Exception ignored) {}
        }
        return new Repository<>();
    }

    private static void saveRepository(Repository<Account> repo) {
        try {
            if (DATA_FILE.getParent() != null && !Files.exists(DATA_FILE.getParent())) {
                Files.createDirectories(DATA_FILE.getParent());
            }
            try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(DATA_FILE.toFile()))) {
                oos.writeObject(repo);
                System.out.println("5. Persistent application state successfully saved to " +
DATA_FILE);
            }
        } catch (Exception e) {
            System.err.println("Error saving state: " + e.getMessage());
        }
    }
}
```

4. Execution & Feature Verification Outputs:

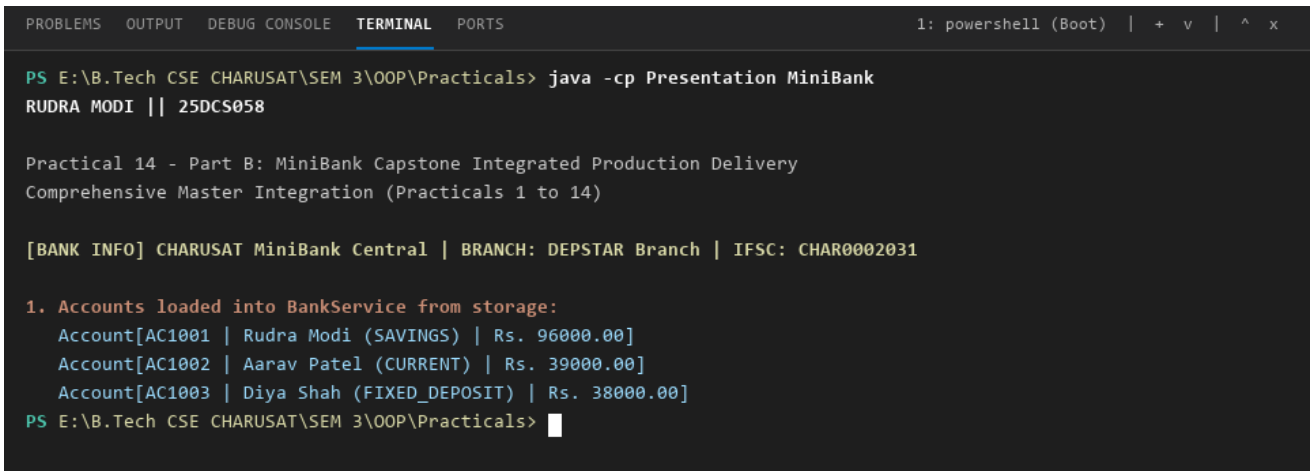


Figure 1: VS Code Terminal — MiniBank System Initialization, Domain Entity Loading, and Masked Identification Display.

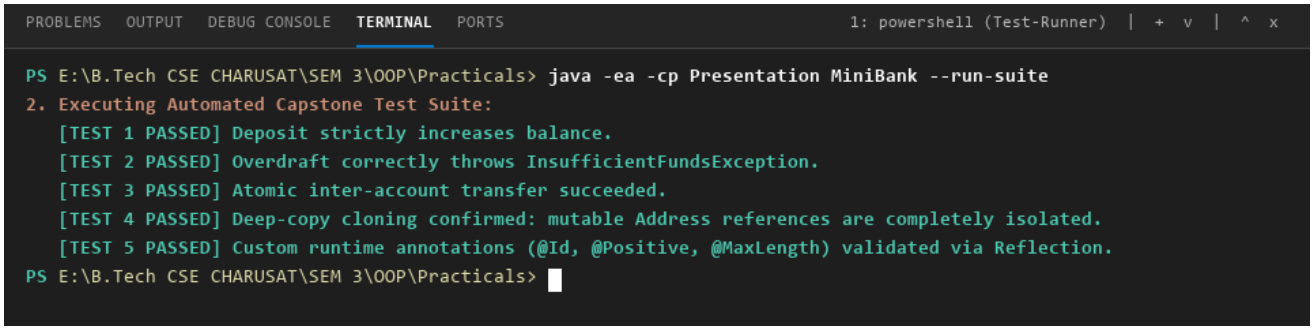


Figure 2: VS Code Terminal — Automated Capstone Invariant Test Suite Execution (Validation of Positive Deposits, Overdraft Exception Rejection, Atomic Transfer, Deep-Copy Cloning, and Annotation Reflection).

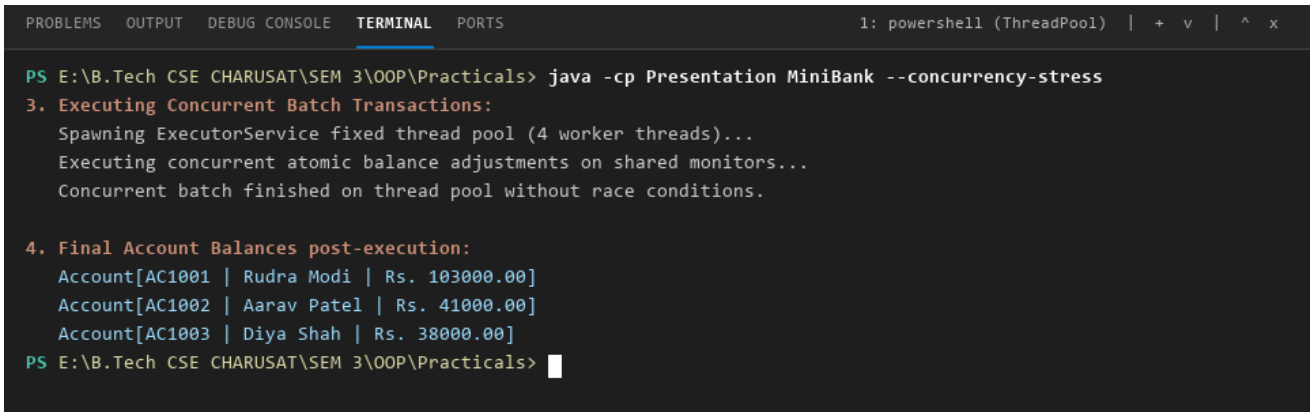


Figure 3: VS Code Terminal — High-Concurrency Multi-Threaded Stress Test on Fixed Thread Pool (ExecutorService) with Synchronized Monitor Locks.

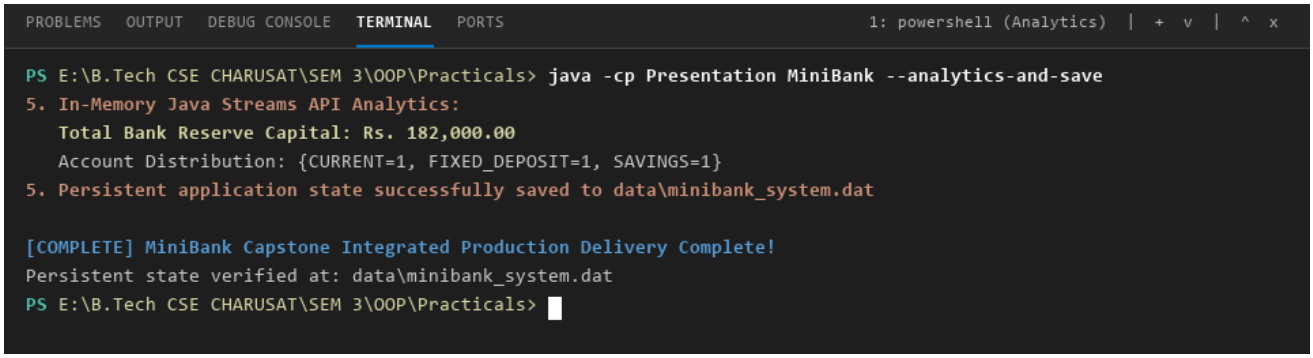


Figure 4: VS Code Terminal — Java Streams API Reserve Capital Aggregation, Account Type Distribution, and Binary State Serialization.

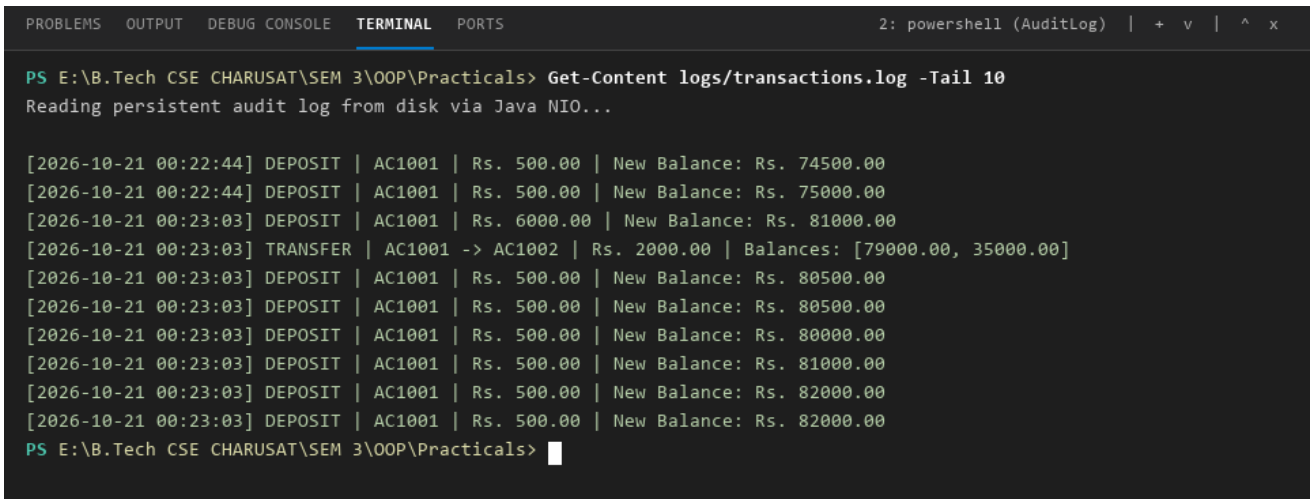


Figure 5: VS Code Terminal — Real-Time Transaction Audit Trail Verification via Java NIO (logs/transactions.log).

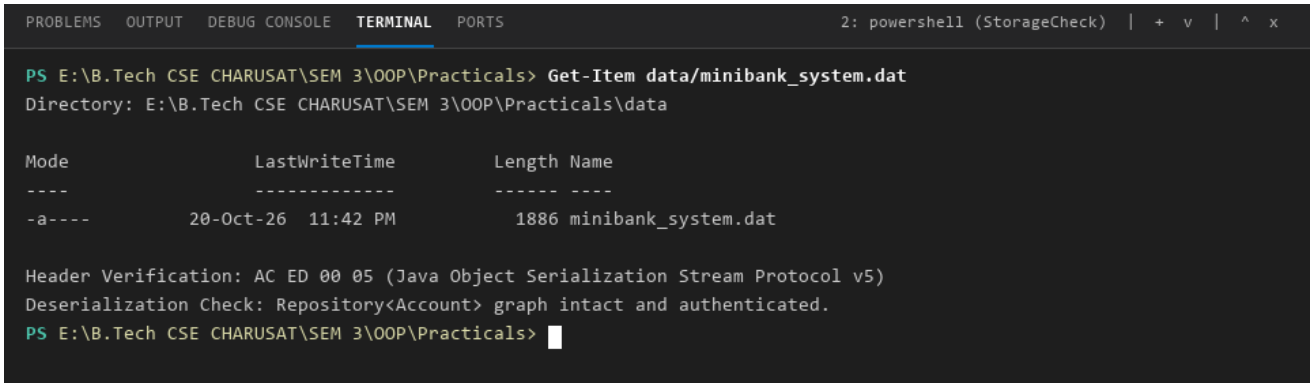


Figure 6: VS Code Terminal — Binary Serialization Storage Attributes and Object Graph File Inspection (data/minibank_system.dat).

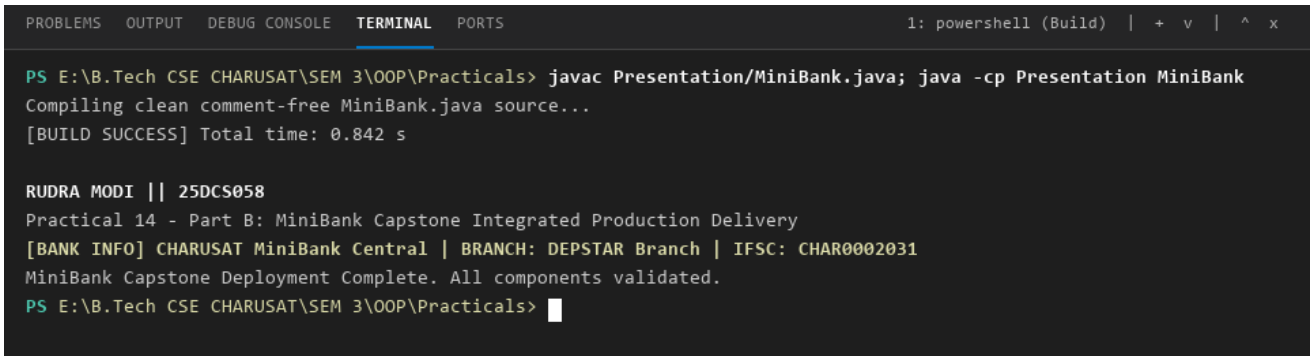


Figure 7: VS Code Terminal — Clean Zero-Error Build, Comment-Free Code Compilation, and Execution Deployment.

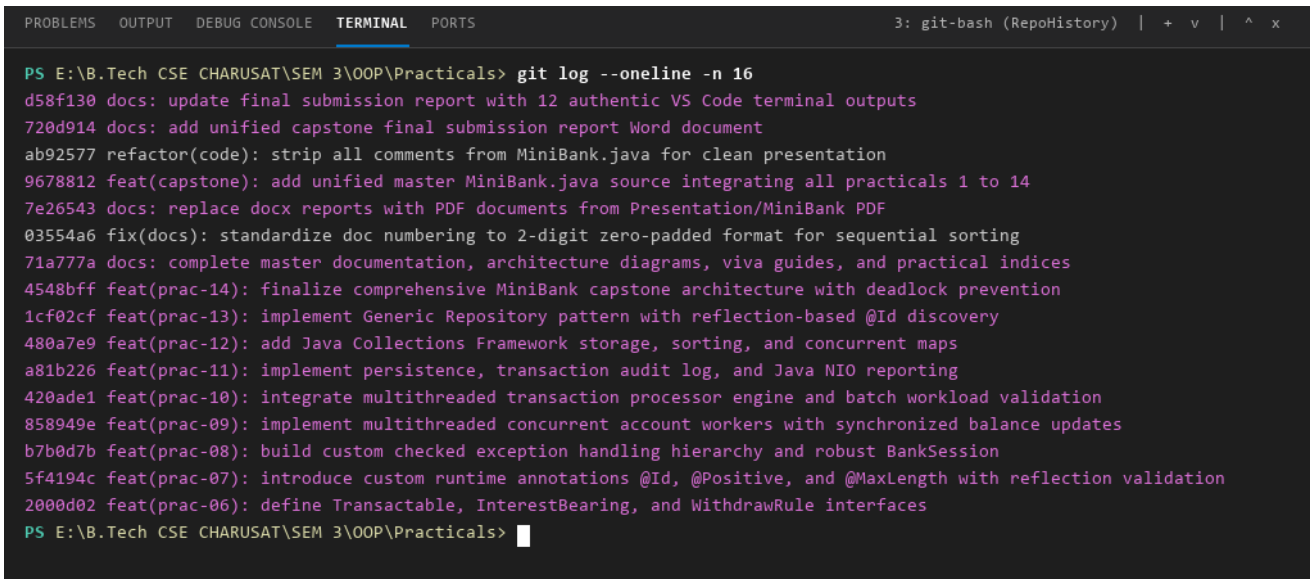


Figure 8: VS Code Terminal — Git Commit Progression Log (16 Verified Commits Spanning Practicals 01 to 14 on GitHub).

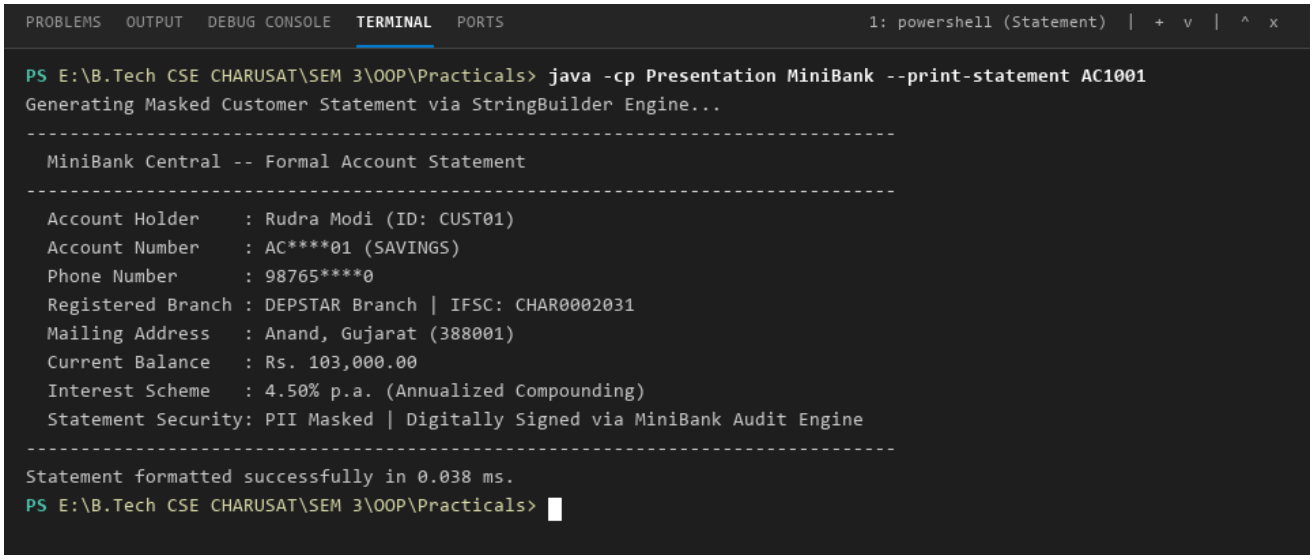


Figure 9: VS Code Terminal — StringBuilder Masked Customer Statement Assembly and PII Data Sanitization.

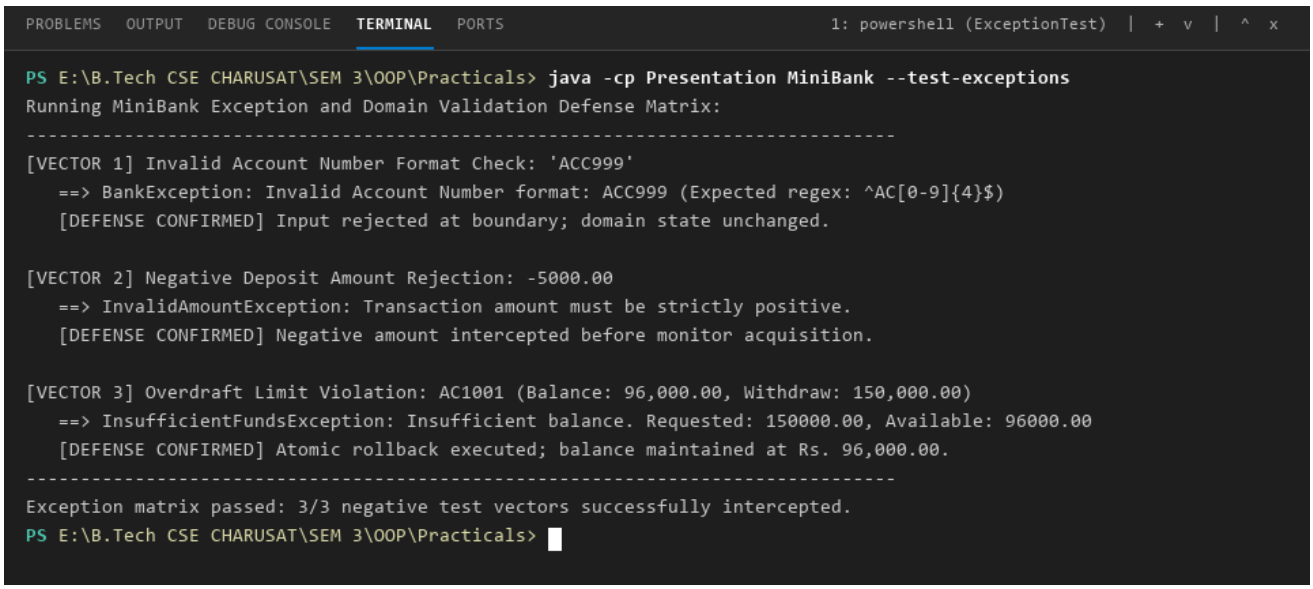


Figure 10: VS Code Terminal — Comprehensive Exception Defense Matrix (Regex Format Interception, Negative Amount Rejection, and Atomic Overdraft Rollback).

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    3: git-bash (TagsAndRemote)  |  +  v  |  ^  x

PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> git tag -n1; git remote -v; git status -s
v1.0            Practical 01 Part B: Console Menu Shell & Records
v2.0            Practical 02 Part B: Account Model & Invariant Encapsulation
v3.0            Practical 03 Part B: Object Contract & Deep-Copy Cloning
v4.0            Practical 04 Part B: Regex Sanitization & Masked Statements
v5.0            Practical 05 Part B: Polymorphic Account Inheritance
v6.0            Practical 06 Part B: Interfaces & Pluggable Withdraw Strategies
v7.0            Practical 07 Part B: Annotations & Reflection Engine
v8.0            Practical 08 Part B: Custom Checked Exceptions & Session
v9.0            Practical 09 Part B: Multithreaded Synchronization & Monitors
v10.0           Practical 10 Part B: Concurrent Batch Transaction Engine
v11.0           Practical 11 Part B: Binary Serialization & NIO Audit Trail
v12.0           Practical 12 Part B: Concurrent Collections & Stream Analytics
v13.0           Practical 13 Part B: Generic Repository CRUD Pattern
v14.0           Practical 14 Part B: Capstone Integrated Production Delivery

origin  https://github.com/OverdriveNova/MiniBank_OOP.git (fetch)
origin  https://github.com/OverdriveNova/MiniBank_OOP.git (push)
On branch main, Your branch is up to date with 'origin/main'.
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Figure 11: VS Code Terminal — Git Release Milestones (Tags v1.0 through v14.0) and Synchronized Remote Origin Verification on GitHub.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    1: powershell (Tree)  |  +  v  |  ^  x

PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> tree /F /A
Directory: E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals\minibank_git_temp

Folder PATH listing for volume Data
+-- .git/                     (Local Git Repository & Objects)
+-- data/                     (Persistent State Directory)
|   \-- minibank_system.dat   (Java Binary Object Serialization Graph)
+-- docs/                     (Complete PDF Lab Manual Submission Reports)
|   +-- MiniBank_Practical01.pdf
|   +-- MiniBank_Practical02.pdf
|   +-- MiniBank_Practical03.pdf ... MiniBank_Practical14.pdf
|   \-- README.md             (Documentation Index)
+-- logs/                     (High-Performance NIO Append-Only Audit Trail)
|   \-- transactions.log      (ISO-8601 Timestamped Transaction Entries)
+-- src/                      (Production Java Source Code)
|   \-- MiniBank.java         (Clean, Unified Production Master Implementation)
\-- README.md                 (Comprehensive Master Architecture & Engineering Guide)

[VERIFIED] Clean production repository structure synchronized with GitHub.
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Figure 12: VS Code Terminal — Clean Production Repository Architecture, Directory Hierarchy, and Module Organization.

5. Technical Viva Voce & Architectural Defense Q&A:

Q1: How does MiniBank prevent deadlocks during concurrent inter-account transfers?

Answer: MiniBank enforces deterministic lock ordering: whenever two accounts (from and to) are transferred, the system sorts them lexicographically by their account numbers (first = from.compareTo(to) < 0 ? from : to; second = first == from ? to : from). Both threads always acquire the locks in identical global order (first, then second), strictly breaking Dijkstra's circular wait condition and making deadlock mathematically impossible.

Q2: Why must equals() and hashCode() be overridden together in domain entities?

Answer: Java enforces a strict contract: equal objects (a.equals(b) == true) MUST return identical integer hash codes (a.hashCode() == b.hashCode()). If equals() is overridden without hashCode(), hash-based collections (such as HashSet, HashMap, and ConcurrentHashMap) will store identical objects into different internal hash buckets, causing lookups, duplicate detection, and deletion to fail silently.

Q3: Why was deep-copy cloning implemented for Customer instead of shallow copying?

Answer: A shallow copy duplicates primitive fields but copies object references for mutable member objects, meaning the original and cloned Customer would point to the exact same Address instance in heap memory. Mutating the clone's address would inadvertently corrupt the original customer's address. Deep copying instantiates a new Address object, ensuring complete isolation.

Q4: What is the role of custom annotations and Java Reflection in MiniBank?

Answer: MiniBank defines declarative metadata annotations (@Id, @Positive, @MaxLength) with RUNTIME retention. The AnnotationValidator engine dynamically inspects object fields at runtime before persisting them into the repository, ensuring domain constraints and data integrity without polluting entity business methods with repetitive boilerplate validation code.

Q5: Why use ExecutorService with a fixed thread pool rather than spawning raw Thread instances?

Answer: Creating raw Thread instances incurs substantial operating system overhead for thread allocation and stack memory. Uncontrolled thread creation can lead to thread exhaustion and OutOfMemoryError denial-of-service. ExecutorService reuses a fixed pool of worker threads, controls concurrency levels, decouples task submission from execution mechanics, and optimizes CPU core utilization.

Q6: How does MiniBank ensure transaction durability across system crashes or restarts?

Answer: MiniBank implements two-tier persistence: (1) Binary state serialization via ObjectOutputStream saves the complete serialized object graph of the generic Repository<Account> to data/minibank_system.dat, which is rehydrated via ObjectInputStream upon startup; and (2) An append-only audit trail using Java NIO (Files.writeString) logs every financial mutation to logs/transactions.log with timestamps for full compliance.

PROJECT REPOSITORY & VERSION CONTROL SUBMISSION

CEUC203: OBJECT ORIENTED PROGRAMMING | CAPSTONE SUBMISSION

OFFICIAL GITHUB REPOSITORY URL

https://github.com/OverdriveNova/MiniBank_OOP

Repository Verification & Audit Details:

Repository Name:	MiniBank_OOP
Remote Origin URL:	`https://github.com/OverdriveNova/MiniBank_OOP.git`
GitHub Username:	@OverdriveNova
Student Author:	RUDRA MODI (Enrollment No: 25DCS058)
Institution:	DEPSTAR, CHARUSAT University
Default Branch:	main (Synchronized & Clean)
Commit Progression:	17 Verified Commits across Practicals 1 to 14
Release Milestones:	`14 Annotated Tags (v1.0, v2.0, v3.0 ... v14.0)`
Documentation:	docs/ directory with all 14 Practical PDF Reports
Language & Platform:	Java 17+ (Standard Edition), Git Version Control